

Note This assignment

Learning Outcomes

After completing this program, students will be able to

- **Use** Scheme *comment stripping filter*, *tokenizer*, and *parser* to translate Scheme programs from text into SExp-based *abstract syntax trees*.
- **Extend** the SExp hierarchy with a `Callable` class representing built-in and `lambda` expressions.
- **Implement** `eval` and `apply` in the SExp hierarchy

Getting Started

Read the **whole** assignment. This is important for *every* assignment: it puts the task into your brain so that it can begin working on answering the questions. Of particular interest when you read are the SLO (first section above); gives you the links between the assignment to the big picture (learning computer science).

Consider drawing a picture (or sketching code) for how `eval` works in the case of `Cons` and how `apply` works in `Callable`.

The REPL

Consider a REPL wrapped around a `Parser` object. Assuming that `next` never calls `hasNext`, the following code should prompt the user for an expression, read and parse the expression, and then print the expression out, all until the `EOFExpression` is returned:

```
Parser in = new Parser(...); // System.in

while (true) {
    try {
        prompt();
        SExp expression = in.next();

        if (expression.equals eofExpression))
            break;
        System.out.println(expression);
    } catch (SchemeSyntaxException syntax) {
        System.error.println(syntax); // nothing to be done for recovery
    } ... other exceptions (like I/O) if necessary
}
```

This is, fundamentally, the solution to program 05, the parser. The biggest change in the REPL is that now, instead of printing `expression` it will print the *evaluation* of `expression`:

```
Parser in = new Parser(...); // System.in
Env global = new GlobalEnv(); // the global Scheme environment

while (true) {
    try {
```

```

prompt();
SExp expression = in.next();

if (expression.equals eofExpression))
    break;
SExp result = expression.eval(global);
System.out.println(result);
} catch (SchemeSyntaxException syntax) {
    System.error.println(syntax); // nothing to be done for recovery
} catch (SchemeTypeException type) {
    System.error.println(type); // nothing to be done for recovery
} catch (SchemeUndefinedSymbolException unknown) {
    System.error.println(unknown); // nothing to be done for recovery
} ... other exceptions (like I/O) if necessary
}

```

eval

What does `SExp.eval(Env env)` do? It depends on the type of the expression:

<number> self-evaluating to the numeric `SExp`

#t, #f self-evaluating to the Boolean `SExp`

<symbol> evaluates to `env.lookup(<symbol>)`

Error: Symbol not found in Env chain that is active.

'<expression> Translated to the expression `(quote <expression>)` and *that* `SExp` is evaluated.

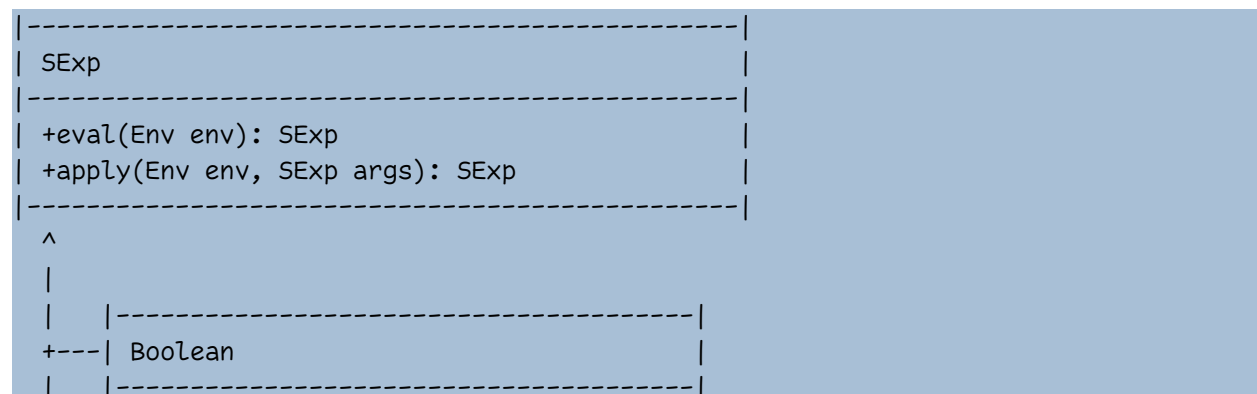
(quote <expression>) Evaluates to exactly `<expression>`.

(define <symbol> <expression>) evaluates `<expression>` in current env into `<result>` and then does `env.define(<symbol>, <result>)`

(<callable-expression> <actual-parameter>*) Any “regular” list entered by the user is broken into its `<car>` and `<cdr>`. The `<car>` is evaluated to a `Callable`. The `<cdr>` is passed to the `Callable`’s `apply` method and the result is returned.

Error: Type error if `<car>` is *not* a `Callable`.

Expression Hierarchy



```

| | +eval(Env env): return this; |
| | -----|
|
| | -----|
+---| Callable |
| | -----| | |
| | +eval(Env env): return this; |
| | +apply(Env env, SExp args): evaluate args (as necessary) |
| | -----|
| | ^ |
| | | | -----|
| | +---| CompoundProcedure |
| | | | -----|
| | | +eval(Env env): return this; |
| | | +apply(Env env, SExp args): evaluate args, local env, body |
| | | -----|
|
| | -----|
+---| Cons |
| | -----|
| | +eval(Env env): return (car.eval(env)).apply(env, cdr); |
| | -----|
|
| | -----|
+---| EOFExpression |
| | -----|
|
| | -----|
+---| Number |
| | -----|
| | +eval(Env env): return this; |
| | -----|
|
| | -----|
+---| Symbol |
| | -----|
| | +eval(Env env): return env.get(this); |
| | -----|

```

Env

The provided Env has lookup/define methods as well as constructors for global and/or threaded environments. `GlobalEnv` is a class that *contains* an Env with the global (built in to Scheme) functions definitions associated with appropriate symbols.

In an Env, a `Symbol` object is associated with an `SExp`. A `Callable` AST is constructed with two Java functions: an argument transformation function, an `Argsmorgrifier`, which takes an Env and a `Cons` and evaluates whatever part of the list needs to be modified, returning the modified results; the second

function is an `Applicable`, a function that takes an `Env` and the *actual*

Each function is stored as a static Java function of two parameters: the *calling* `Env` and the `Cons` containing the *actual* parameters.

Consider the evaluation of the expression

```
(+ (* 5 4) 7)
```

To avoid getting too far into the weeds we will focus on just handling calling `+` and pretend that “magic happens” when the multiplication must be evaluated.

`Cons.eval(env)` breaks the `Cons` into its `car`, the `Symbol` `+`, and its `car`, the `Cons` `(( * 5 4) 7)`. Call the two parts `f` (for function, the `car`) and `actual` (for actual parameters, the `cdr`).

Call `f.eval(env)`. It should return something `Callable`. In the global environment provided, the definition of `+` is:

```
entry(Symbol("+"),
  Callable(Callable::evalAll,
    (callingEnv, actualParameterCons) ->
      new Number(actualParameterCons.nth(0).toNumber().value() +
        actualParameterCons.nth(1).toNumber().value())))
```

When a `Callable` is constructed, the first parameter is the evaluation strategy for the actual parameters. Taking the name as given, it evaluates all of the parameters. That means that the `actualParameterCons` will be

```
Callable::evalAll(actual)
```

That means we have

```
(( * 5 4) 7) => (20 7)
```

(Remember: magic evaluated the multiply and `Numbers` are self-evaluating (they are their own evaluation).)

The second parameter in constructing the `Callable` is the function to apply to the actual parameters. Notice that it takes an environment (the calling environment) and the modified actual parameter list. Here it takes the actual parameter list and pulls out (into Java) the numeric values of the first and second elements. Java then *adds* them (something Java knows how to do), constructs a `Number` AST from the result, and returns that as the result of applying the function.

Thus:

```
(+ (* 5 4) 7) => 27
```

Note that the `Global` you are given has one sample built-in of most types. You are expected to build the others. So you get `+`, `-`, `*`, and `/` are pretty much the same. As with comparison operators other than `=`, Boolean functions other than `null?`, and so on.

The `car`, `cdr`, and `cons` built-ins should work with your AST. You might not have written the `nth(n)` method for `Cons`, or some of the other methods used in the provided code. You can either write them or write something equivalent.

Turn In

This lab’s **repo name** is `p05-MakeUAScheme`

- Go to the `S25-443-<CCID>` organization page. You need to create a git repo on Gitea and associate it with the local repo you have used for this lab.

- Click the *New Repository* button, upper right of the organization page.
- As stated at the top of the lab and the top of this section, this lab's **repo name** is p05-MakeUAScheme. The name is fixed so that downloading it can be scripted.¹
The rest of the fields on the repository creation page can be left at their default values. Click the *Create Repository* button.
- After creating the repository, Gitea will show an URL field and two lists of commands. Make sure the *HTTPS* button next to the URL is clicked. Then copy, paste, and execute the two commands for *Pushing an existing repository from the command line* in the lab01 root of the working directory.
- Reload the repository in your browser and you should see the files in the repo that you submitted.

¹The name **on Gitea** has nothing to do with the name in your files.